

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

AIM: Feed Forward Backpropagation Neural Network

a) Implement the Feed Forward Backpropagation algorithm to train a neural network.

b) Use a given dataset to train the neural network for a specific task.

c) Evaluate the performance of the trained network on test data.

Input

```
Feed Forward Backpropagation.py - C:/Users/DELL/Downloads/Feed Forward Backpropagation.py (3.13.11)
File Edit Format Run Options Window Help

# Feed Forward Backpropagation Neural Network

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load dataset
data = pd.read_csv("conn4_log_labeled.csv", low_memory=False)

print("Dataset loaded successfully")
print("Dataset shape:", data.shape)

# Clean column names
data.columns = data.columns.str.strip()

# Find the column containing Benign and Malicious
target_column = None

for column in data.columns:
    values = data[column].astype(str).str.strip().str.lower()

    if "benign" in values and "malicious" in values:
        target_column = column
        break

if target_column is None:
    print("\nBenign/Malicious column was not found.")
    print("\nColumns in dataset:")
    print(data.columns)
    exit()

print("\nTarget column found:", target_column)

# Clean target values
data[target_column] = (
    data[target_column]
    .astype(str)
    .str.strip()
    .str.lower()
)

# Keep only Benign and Malicious
data = data[
    data[target_column].isin(["benign", "malicious"])
].copy()

print("\nClass distribution:")
print(data[target_column].value_counts())

# Convert target to numbers
data["target"] = data[target_column].map({
    "benign": 0,
    "malicious": 1
})

# Select numerical features
X = data.select_dtypes(include=["number"]).copy()

# Remove target
X = X.drop(columns=["target"], errors="ignore")

y = data["target"]

# Replace infinite values
X = X.replace([np.inf, -np.inf], np.nan)

# Fill missing values
X = X.fillna(0)
```

Soham Patil

Tycs T103

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
Feed Forward Backpropagation.py - C:/Users/DELL/Downloads/Feed Forward Backpropagation.py (3.13.11)
File Edit Format Run Options Window Help

# Remove columns with only one value
X = X.loc[:, X.nunique() > 1]

print("\nNumber of features:", X.shape[1])

# Balance dataset
temp = X.copy()
temp["target"] = y.values

benign = temp[temp["target"] == 0]
malicious = temp[temp["target"] == 1]

print("\nBenign records:", len(benign))
print("\nMalicious records:", len(malicious))

n = min(len(benign), len(malicious), 4000)

if n < 2:
    print("\nNot enough records for training.")
    exit()

benign = benign.sample(n=n, random_state=42)
malicious = malicious.sample(n=n, random_state=42)

balanced = pd.concat([benign, malicious])

balanced = balanced.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

# Separate features and target
X = balanced.drop(columns=["target"])
y = balanced["target"]

print("\nFinal dataset:")
print("Benign:", sum(y == 0))

Ln: 182 Col: 0

Feed Forward Backpropagation.py - C:/Users/DELL/Downloads/Feed Forward Backpropagation.py (3.13.11)
File Edit Format Run Options Window Help

print("Benign:", sum(y == 0))
print("Malicious:", sum(y == 1))
print("Total:", len(y))

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining records:", len(X_train))
print("\nTesting records:", len(X_test))

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create neural network
model = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    activation="relu",
    solver="adam",
    max_iter=100,
    random_state=42
)

# Train neural network
print("\nTraining neural network...")

model.fit(X_train, y_train)

print("Training completed")

Ln: 182 Col: 0
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
Feed Forward Backpropagation.py - C:/Users/DELU/Downloads/Feed Forward Backpropagation.py (3.13.11)
File Edit Format Run Options Window Help
model.fit(X_train, y_train)

print("Training completed")

# Predict test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(round(accuracy * 100, 2), "%")

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Benign", "Malignant"]
    )
)

# Training loss graph
plt.plot(model.loss_curve_)

plt.title("Training Loss")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid()

plt.show()
```

Output

```
"IDLE Shell 3.13.11"
File Edit Shell Debug Options Window Help
Dataset loaded successfully
Dataset shape: (156104, 24)

Target column found: set[string]

Class distribution:
set[string]
malignant 151567
benign 4536
Name: count, dtype: int64

Number of features: 6

Benign records: 4536
Malignant records: 151567

Final dataset:
Benign: 4000
Malignant: 4000
Total: 8000

Training records: 6400
Testing records: 1600

Training neural network...
Training completed

Accuracy:
99.25 %

Confusion Matrix:
[[794 6]
 [ 6 794]]

Classification Report:
precision recall f1-score support
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

